文档：

官网：https://etcd.io/docs/v3.5/
源码：https://github.com/etcd-io/etcd
下载地址：https://github.com/etcd-io/etcd/releases
docker镜像：https://quay.io/repository/coreos/etcd?tab=tags&tag=latest
etcd部署：http://play.etcd.io/install
etcd/client/v3：https://github.com/etcd-io/etcd/tree/main/client/v3

etcd是什么

etcd 是一个分布式键值对存储，设计用来**可靠而快速的保存关键数据**并提供访问。通过分布式锁，leader选举保障可靠的分布式协同。

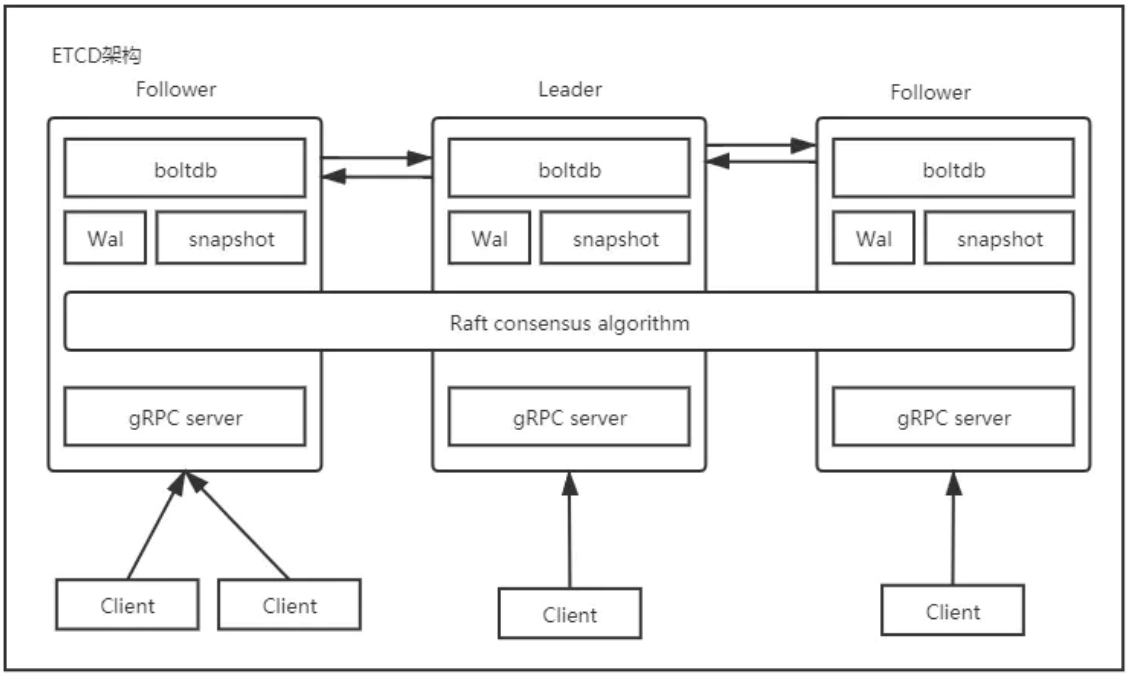
etcd 特点

1. 完全复制，集群中的每个节点均拥有全量数据
2. 强一致性，etcd通过raft共识算法保证集群数据的一致性
3. 数据结构简单，仅字符串格式
4. 可存储的数据量较少
5. 每个节点支持1000次/秒的写操作，2000次/秒的读操作

使用场景

1. 服务注册与服务发现
2. 分布式配置中心
3. 分布式锁
4. 分布式选主
5. 读多写少的场景

架构图



1. Leader：Raft算法中通过竞选而产生的处理所有数据提交的节点
2. Follower：竞选失败的节点作为Raft中的从属节点，为算法提供强一致性保证
3. Candidate：当Follower超过一定时间接收不到Leader的心跳时转变为Candidate开始竞选
4. Learner：etcd集群中的只读节点，拥有全量数据，不参与Leader的竞选
5. BoltDB：一个支持事务的KV存储引擎，用于对ETCD数据的持久化，支持并发读事务，串行写事务
6. WAL 预写式日志：etcd中的数据提交都会先记录到日志，日志成功后才会做数据持久化
7. Snapshot，快照：防止WAL日志过多，用于存储某一时刻etcd的所有数据，可用于节点故障恢复
8. gRPC Server：节点与节点，节点与客户端通过gRPC 通信和信息同步，每个节点都是一个gRPC 服务器
9. Raft：共识算法，etcd数据强一致性的保障
10. Client：etcd客户端，可以是命令行客户端 etcdctl ，也可以是各个语言版本的client库，可通过grpc与http与server端通信

基本原理

MVCC（多版本并发控制）

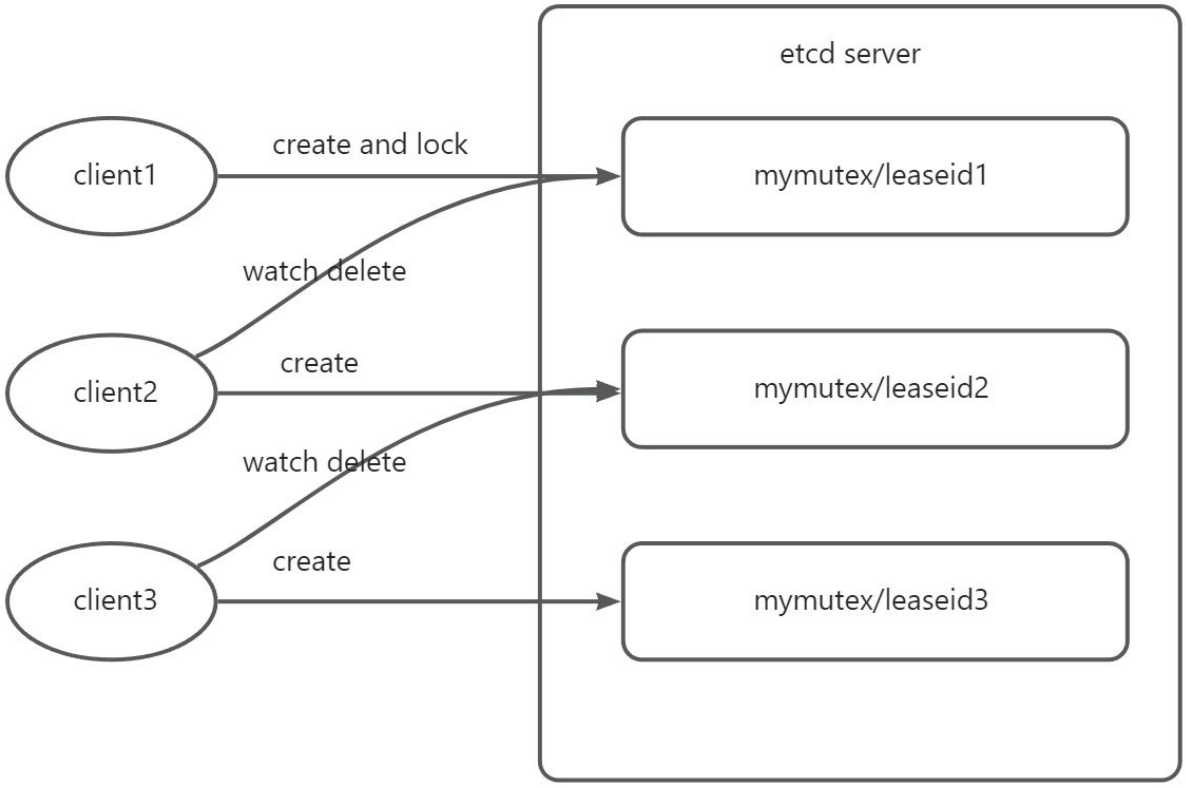
1. 作用

MVCC即在对数据修改时，不是去改变数据的值，而是新增一条数据记录，数据的版本号递增1。在读取数据时实际上是读取到当前版本的数据记录。MVCC旨在提高数据库的并发性能，更好的解决读写之间的冲突，从而实现读写冲突的情况下，不加锁并发读。

2. 数据版本号：

revision：etcd键空间版本号，key发生变更，则revision加1；全局单调递增，64bits
create_revision： 创建数据时，对应的revision
mod_revision： 数据修改时，对应的revision
version： 表示当前key被修改的次数

分布式锁



1. 锁的持有者为：前缀为 mymutex 且创建时间最早的key，即根据create version 正序第一个key值
2. etcd中任何数据变化都会导致全局的revision发生递增
3. 客户端向etcd发送事务，事务内容：1.向etcd添加当前客户端的键值对，并获得结果；2.向etcd中获取目前的锁持有者。
4. 如果不存在锁持有着，则自身即为持有者；如果锁持有者的create version 等于当前自身的create version则自身即为持有者
5. 如果自身不是持有者，则监听前缀为 mymutex 且 create version 比自身小一个版本号的key的删除事件，一旦删除，则自身获取到锁

数据写流程

1. 只有leader节点能处理写流程，当follower节点收到写请求后会转发到leader
2. 将请求提案广播到各个follower节点，并记录WAL，当大多数节点成功记录WAL之后，该请求提案的状态将变成已提交状态。
3. 由各个节点根据WAL应用数据，对数据进行持久化
4. 响应写请求，返回处理结果

读流程

线性读：

1. etcd 默认读模式是线性读，在延时和吞吐量上相比串行读略差一点，适用于对数据一致性要求高的场景
2. 当收到一个线性读请求时，首先会从 Leader 获取集群最新的已提交的日志索引。
3. Leader 收到 ReadIndex 请求时，为防止脑裂等异常场景，会向 Follower 节点发送心跳确认，一半以上节点确认 Leader 身份后才能将已提交的索引 (committed index) 返回给节点。
4. 节点则会等待，直到状态机已应用索引 (applied index) 大于等于 Leader 的已提交索引时 (committed Index)，然后去通知读请求。

串行读：

1. 节点数据返回、无需通过 Raft 协议与集群进行交互。它具有低延时、高吞吐量的特点，适合对数据一致性要求不高的场景

单机安装

linux

```
ETCD_VER=v3.5.7

GITHUB_URL=https://github.com/etcd-io/etcd/releases/download
DOWNLOAD_URL=${GITHUB_URL}

rm -f /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz
rm -rf /tmp/etcd-download-test && mkdir -p /tmp/etcd-download-test

curl -L ${DOWNLOAD_URL}/${ETCD_VER}/etcd-${ETCD_VER}-linux-amd64.tar.gz -o
/tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz
tar xzvf /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz -C /tmp/etcd-download-test --
strip-components=1
rm -f /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz

/tmp/etcd-download-test/etcd --version
/tmp/etcd-download-test/etcdctl version
/tmp/etcd-download-test/etcdutl version
```

```
# start a local etcd server
/tmp/etcd-download-test/etcd

# write,read to etcd
/tmp/etcd-download-test/etcdctl --endpoints=localhost:2379 put foo bar
/tmp/etcd-download-test/etcdctl --endpoints=localhost:2379 get foo
```

docker

```
rm -rf /tmp/etcd-data.tmp && mkdir -p /tmp/etcd-data.tmp && \
  docker rmi quay.io/coreos/etcd:v3.5.7 || true && \
```

```
  docker run \
  -p 2379:2379 \
  -p 2380:2380 \
  --mount type=bind,source=/tmp/etcd-data.tmp,destination=/etcd-data \
  --name etcd-gcr-v3.5.7 \
  quay.io/coreos/etcd:v3.5.7 \
  /usr/local/bin/etcd \
  --name s1 \
  --data-dir /etcd-data \
  --listen-client-urls http://0.0.0.0:2379 \
  --advertise-client-urls http://0.0.0.0:2379 \
  --listen-peer-urls http://0.0.0.0:2380 \
  --initial-advertise-peer-urls http://0.0.0.0:2380 \
  --initial-cluster s1=http://0.0.0.0:2380 \
  --initial-cluster-token tkn \
  --initial-cluster-state new \
  --log-level info \
  --logger zap \
  --log-outputs stderr

docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcd --version
docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcdctl version
docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcdutl version
docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcdctl endpoint health
docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcdctl put foo bar
docker exec etcd-gcr-v3.5.7 /usr/local/bin/etcdctl get foo
```

ETCD选项说明

1. etcd 集群运行时会同时监听2个端口，一个是客户端端口（如：2379）用于给客户端连接，一个是集群成员之间（或伙伴，或 peer）通讯的端口（如：2380）。
2. etcd 集群配置的参数名称有一定规则，一般 --initial 前缀标记用于静态部署时启动集群，--discovery 前缀标记在使用发现服务的方式启动集群
3. etcd 集群的安全从两个方面维护，一个是服务端集群成员（peer）之间通信使用 TLS 安全认证，另一个是客户端（etcdctl）访问 etcd 集群使用 TLS 安全认证。
4. TLS 安全认证有一下几个文件组成

- CA 证书：ca-file
- 私钥文件： key-file
- 证书文件: cert-file
- 证书吊销列表： crl-file

命令行选项

```
Member:
  # 节点名称
  --name 'default'
  # 数据存储目录
  --data-dir '${name}.etcd'
  # 预写式日志存储目录
  --wal-dir ''
```

```
  # 触发磁盘快照的提交事务数
  # 当 etcd 的事务数量达到 --snapshot-count 配置值时，etcd 将会自动触发快照操作。
  # 这有助于减少故障恢复时间，并限制日志文件的大小，以避免无限增长
  --snapshot-count '100000'
  # 心跳间隔的时间（毫秒）
  --heartbeat-interval '100'
  # 选主超时时间（毫秒）
  --election-timeout '1000'
  # 是否加快初次选主的速度
  # 在 etcd 中，选举是通过 Raft 算法实现的，在启动一个新的 etcd 节点时，它需要与集群中的其他节点进行通信，并参与到选举过程中。
  # initial-election-tick-advance 可以让新节点在加入集群后立即参与选举，而不需要等待一定时间才能参与
  --initial-election-tick-advance 'true'
  # 监听节点流量的地址列表
  --listen-peer-urls 'http://localhost:2380'
  # 监听客户端流量的地址列表
  --listen-client-urls 'http://localhost:2379'
  # 保留的快照文件的最大数量，0表示不限制
  --max-snapshots '5'
  # 保留的预写式日志文件的数量，0表示不限制
  --max-wals '5'
  # 设置空间配额，当占用空间超过指定配额则引发警告。0表示默认空间配额,一般为2GB。单位为字节
  --quota-backend-bytes '0'
  # 指定BoltDB使用的空闲列表的类型，支持array和map
  # array 适合小数据集和低并发场景
  # map 适合大数据集和高并发场景
  --backend-bbolt-freelist-type 'map'
  # 定义了后端批量操作的间隔时间
  --backend-batch-interval ''
  # 用来指定每个批处理操作中最大的请求数量
  --backend-batch-limit '0'
  # 事务中允许的最大操作数
  --max-txn-ops '128'
  # 服务器将接受的最大客户端请求大小（字节）
  --max-request-bytes '1572864'
  # 每个客户端一次可以打开的最大并发流
  --max-concurrent-streams 'math.MaxUint32'
  # 客户端在ping服务器之前应等待的最短持续时间间隔
  --grpc-keepalive-min-time '5s'
  #  服务端ping客户端的频率，检查连接是否处于活动状态（0表示禁用）
  --grpc-keepalive-interval '2h'
  # 关闭非响应连接之前的额外等待时间（0表示禁用）
  --grpc-keepalive-timeout '20s'
  # 在监听器上设置套接字选项 SO_REUSEPORT，允许重新绑定已在使用的端口
  --socket-reuse-port 'false'
  # 在监听器上设置套接字选项 SO_REUSEADDR，允许绑定到TIME_WAIT状态下的地址
  --socket-reuse-address 'false'

Clustering:
  # 向集群的其他节点发布此节点的访问地址
  --initial-advertise-peer-urls 'http://localhost:2380'
  # 集群初始成员配置，是etcd静态部署的核心初始化配置，
    # 它说明了当前集群由哪些URLs组成,此处default为节点名称
  --initial-cluster 'default=http://localhost:2380'
  # 初始化集群状态（new 或 existing）
```

```
    # 在新建集群时，无论是静态部署或是动态发现部署，所有成员都应该设置为 new
    # 如果集群在运行过程节点信息发生变化，
    # 比如在已有的集群中新加节点时，新节点的配置应该设置为 existing
  --initial-cluster-state 'new'
  # 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
  --initial-cluster-token 'etcd-cluster'
  # 对外发布此节点的客户端访问地址
  --advertise-client-urls 'http://localhost:2379'
  # etcd 有两种部署方式，已知peer成员列表的静态部署，未知peer成员的动态服务发现部署。
    # 该参数用于动态发现部署
  --discovery ''
  # 服务发现失败时预期行为（"exit"或"proxy"），proxy 仅支持v2版本
  --discovery-fallback 'proxy'
  # 用于到发现服务的流量的HTTP代理
  --discovery-proxy ''
  # 用于引导群集的DNS srv域
  --discovery-srv ''
  # 引导时查询的dns srv名称的后缀
  --discovery-srv-name ''
  # 拒绝将导致法定人数丢失的重配置请求
  --strict-reconfig-check 'true'
  # 启用以运行额外的Raft选举阶段
  --pre-vote 'true'
  # 自动压缩保留时长或版本数，0表示禁用
  --auto-compaction-retention '0'
  # 自动压缩模式：periodic 表示基于时长，默认单位为小时(h);revision 表示基于版本保留
  --auto-compaction-mode 'periodic'

Security:
  # 客户端-服务器TLS证书文件的路径
    # 客户端访问集群所依赖的证书文件路径
  --cert-file ''
  # 客户端-服务器TLS密钥文件的路径
  --key-file ''
  # 启用客户端证书身份验证
  --client-cert-auth 'false'
  # 客户端证书吊销列表文件的路径
  --client-crl-file ''
  # 客户端证书身份验证允许的TLS主机名
  --client-cert-allowed-hostname ''
  # 客户端-服务器TLS受信任CA证书文件的路径。
  --trusted-ca-file ''
  # 客户端TLS 使用生成的证书
  --auto-tls 'false'
  # 节点-节点 TLS证书文件的路径。
  --peer-cert-file ''
  # 节点-节点 TLS密钥文件的路径
  --peer-key-file ''
  # 启用节点间证书身份认证
  --peer-client-cert-auth 'false'
  # 节点-节点 TLS受信任CA文件的路径
  --peer-trusted-ca-file ''
  # 节点-节点证书连接允许的CN
  --peer-cert-allowed-cn ''
  # 节点-节点 TLS证书允许的主机名
  --peer-cert-allowed-hostname ''
```

```
  # 如果 --peer-key-file 和 --peer-cert-file 未提供，则使用自动生成证书
  --peer-auto-tls 'false'
  # 指定ClientAutoTLS和PeerAutoTLS时，
  # etcd自动生成的客户端和成员证书的有效期，单位为年，默认值为1
  --self-signed-cert-validity '1'
  # 成员证书吊销列表文件的路径
  --peer-crl-file ''
  # 客户端/服务器和成员端之间支持的TLS密码套件的逗号分隔列表
  --cipher-suites ''
  # 逗号分隔的origin白名单，用于跨origin资源共享,(空或者*表示允许所有)
  --cors '*'
  # HTTP客户端请求中可接受的主机名（空或*表示全部允许）
  --host-whitelist '*'

Auth:
  # 指定v3身份验证令牌类型及其选项（'simple'或'jwt'）
  --auth-token 'simple'
  # 指定用于哈希验证密码的bcrypt算法的成本/强度。有效值介于4和31之间
  --bcrypt-cost 10
  # 指定 auth-token 的有效时长（秒）
  --auth-token-ttl 300

Profiling and Monitoring:
  # 通过HTTP服务器启用运行时分析数据，地址位于客户端URL+“/debug/pprof/”
  --enable-pprof 'false'
  # 设置导出指标的详细级别，设置为：extensive则包括服务器端grpc直方图指标
  --metrics 'basic'
  # 侦听指标和运行状况端点的URL列表
  --listen-metrics-urls ''

Logging:
  # 当前仅支持结构化日志记录的“zap”
  --logger 'zap'
  # 指定日志输出位置，指定 stdout或stderr则跳过日志打印
  --log-outputs 'default'
  # 日志打印级别，支持 debug, info, warn, error, panic, or fatal.
  --log-level 'info'
  # 启用日志轮换（切分）
  --enable-log-rotation 'false'
  # 设置日志轮换配置，JSON配置 MaxSize（MB）、MaxAge（天，0=无限制）、MaxBackups（0=无上限）、LocalTime（使用计算机本地时间）、Compress（gzip）
  --log-rotation-config-json '{"maxsize": 100, "maxage": 0, "maxbackups": 0, "localtime": false, "compress": false}'

Unsafe feature:
  # 强制创建新的单成员群集
  --force-new-cluster 'false'
  # 禁用fsync（不安全）会导致数据丢失
  --unsafe-no-fsync 'false'
```

配置文件选项

```
#Member
# 节点名称
```

```
name: 'default'
# 数据存储目录
data-dir: '/tmp/etcd_data'
# 预写式日志存储目录
wal-dir: '/tmp/etcd_wal'
# 触发磁盘快照的提交事务数
snapshot-count: 100000
# 心跳间隔的时间（毫秒）
heartbeat-interval: 100
# 选主超时时间（毫秒）
election-timeout: 1000
# 是否加快初次选主的速度
initial-election-tick-advance: true
# 监听节点流量的地址列表
listen-peer-urls: 'http://localhost:2380'
# 监听客户端流量的地址列表
listen-client-urls: 'http://localhost:2379'
# 保留的快照文件的最大数量，0表示不限制
max-snapshots: 5
# 保留的预写式日志文件的数量，0表示不限制
max-wals: 5
# 设置空间配额，当占用空间超过指定配额则引发警告。0表示默认空间配额,一般为2GB。单位为字节
quota-backend-bytes: 0
# 指定BoltDB使用的自由列表类型，支持array和map
backend-bbolt-freelist-type: 'map'
# 提交后端事务之前的最长时间
backend-batch-interval: 0
# 提交后端事务之前的最大操作数
backend-batch-limit: 0
# 事务中允许的最大操作数
max-txn-ops: 128
# 服务器将接受的最大客户端请求大小（字节）
max-request-bytes: 1572864
# 每个客户端一次可以打开的最大并发流
max-concurrent-streams: 10
# 客户端在ping服务器之前应等待的最短持续时间间隔
grpc-keepalive-min-time: 5
#  服务端ping客户端的频率，检查连接是否处于活动状态（0表示禁用）
grpc-keepalive-interval: 7200
# 关闭非响应连接之前的额外等待时间（0表示禁用）
grpc-keepalive-timeout: 20
# 在监听器上设置套接字选项 SO_REUSEPORT，允许重新绑定已在使用的端口
socket-reuse-port: false
# 在监听器上设置套接字选项 SO_REUSEADDR，允许绑定到TIME_WAIT状态下的地址
socket-reuse-address: false

# Clustering
# 向集群的其他节点发布此节点的访问地址
initial-advertise-peer-urls: 'http://localhost:2380'
# 集群初始成员配置，是etcd静态部署的核心初始化配置，
# 它说明了当前集群由哪些URLs组成,此处default为节点名称
initial-cluster: 'default=http://localhost:2380'
# 初始化集群状态（new 或 existing）
# 在新建集群时，无论是静态部署或是动态发现部署，所有成员都应该设置为 new
# 如果集群在运行过程节点信息发生变化，
# 比如在已有的集群中新加节点时，新节点的配置应该设置为 existing
```

```
initial-cluster-state: 'new'
# 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
initial-cluster-token: 'etcd-cluster'
# 对外发布此节点的客户端访问地址
advertise-client-urls: 'http://localhost:2379'
# etcd 有两种部署方式，已知peer成员列表的静态部署，未知peer成员的动态服务发现部署。
# 该参数用于动态发现部署
discovery: ''
# 服务发现失败时预期行为（"exit"或"proxy"），proxy 仅支持v2版本
discovery-fallback: 'proxy'
# 用于到发现服务的流量的HTTP代理
discovery-proxy: ''
# 用于引导群集的DNS srv域
discovery-srv: ''
# 引导时查询的dns srv名称的后缀
discovery-srv-name: ''
# 拒绝将导致法定人数丢失的重配置请求
strict-reconfig-check: true
# 启用以运行额外的Raft选举阶段
pre-vote: true
# 自动压缩保留时长或版本数，0表示禁用
auto-compaction-retention: '0'
# 自动压缩模式：periodic 表示基于时长，默认单位为小时(h);revision 表示基于版本保留
auto-compaction-mode: 'periodic'

# Security
# 客户端-服务器TLS证书文件的路径
# 客户端访问集群所依赖的证书文件路径
cert-file: ''
# 客户端-服务器TLS密钥文件的路径
key-file: ''
# 启用客户端证书身份验证
client-cert-auth: 'false'
# 客户端证书吊销列表文件的路径
client-crl-file: ''
# 客户端证书身份验证允许的TLS主机名
client-cert-allowed-hostname: ''
# 客户端-服务器TLS受信任CA证书文件的路径。
trusted-ca-file: ''
# 客户端TLS 使用生成的证书
auto-tls: 'false'
# 节点-节点 TLS证书文件的路径。
peer-cert-file: ''
# 节点-节点 TLS密钥文件的路径
peer-key-file: ''
# 启用节点间证书身份认证
peer-client-cert-auth: 'false'
# 节点-节点 TLS受信任CA文件的路径
peer-trusted-ca-file: ''
# 节点-节点证书连接允许的CN
peer-cert-allowed-cn: ''
# 节点-节点 TLS证书允许的主机名
peer-cert-allowed-hostname: ''
# 如果 peer-key-file 和 peer-cert-file 未提供，则使用自动生成证书
peer-auto-tls: 'false'
# 指定ClientAutoTLS和PeerAutoTLS时，
```

```
# etcd自动生成的客户端和成员证书的有效期，单位为年，默认值为1
self-signed-cert-validity: 1
# 成员证书吊销列表文件的路径
peer-crl-file: ''
# 客户端/服务器和成员端之间支持的TLS密码套件的逗号分隔列表
cipher-suites: []
# 逗号分隔的origin白名单，用于跨origin资源共享,(空或者*表示允许所有)
cors: '*'
# HTTP客户端请求中可接受的主机名（空或*表示全部允许）
host-whitelist: '*'

# Auth
# 指定v3身份验证令牌类型及其选项（'simple'或'jwt'）
auth-token: 'simple'
# 指定用于哈希验证密码的bcrypt算法的成本/强度。有效值介于4和31之间
bcrypt-cost: 10
# 指定 auth-token 的有效时长（秒）
auth-token-ttl: 300

# Profiling and Monitoring
# 通过HTTP服务器启用运行时分析数据，地址位于客户端URL+“/debug/pprof/”
enable-pprof: false
# 设置导出指标的详细级别，设置为：extensive则包括服务器端grpc直方图指标
metrics: 'basic'
# 侦听指标和运行状况端点的URL列表
listen-metrics-urls: ''

# Logging
# 当前仅支持结构化日志记录的“zap”
logger: 'zap'
# 指定日志输出位置，指定 stdout或stderr则跳过日志打印
log-outputs:
- 'default'
# 日志打印级别，支持 debug, info, warn, error, panic, or fatal.
log-level: 'info'
# 启用日志轮换（切分）
enable-log-rotation: false
# 设置日志轮换配置，JSON配置 MaxSize（MB）、MaxAge（天，0=无限制）、MaxBackups（0=无上限）、LocalTime（使用计算机本地时间）、Compress（gzip）
log-rotation-config-json: '{"maxsize": 100, "maxage": 0, "maxbackups": 0, "localtime": false, "compress": false}'

# Unsafe feature
# 强制创建新的单成员群集
force-new-cluster: false
# 禁用fsync（不安全）会导致数据丢失
unsafe-no-fsync: false
```

集群安装（docker）

镜像拉取

```
docker pull quay.io/coreos/etcd:v3.5.7
```

配置文件准备

配置文件存放路径：/home/nick/work/etcdconf/

```
# 节点名称
name: "etcdnode0"
# 数据存储目录
data-dir: "/etcd-data/data"
# 预写式日志存储目录
wal-dir: "/etcd-data/wal"
# 集群成员之间通讯使用URL
listen-peer-urls: "http://0.0.0.0:2380"
# 集群提供给外部客户端访问的URL，即外部客户端必须通过指定的IP加端口访问etcd
listen-client-urls: "http://0.0.0.0:2379"

#集群配置
initial-advertise-peer-urls: "http://192.168.239.149:2380"
# 集群初始成员配置，是etcd静态部署的核心初始化配置，它说明了当前集群由哪些URLs组成,此处default为节点名称
initial-cluster:
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etcdnode2=http://192.168.239.149:22380"
# 初始化集群状态（new 或 existing）
initial-cluster-state: "new"
# 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
initial-cluster-token: "etcd-cluster"
# 向客户端发布的服务端点
advertise-client-urls: "http://192.168.239.149:2379"
logger: "zap"
# 配置日志级别,仅支持  debug, info, warn, error, panic, or fatal
log-level: "warn"
log-outputs:
  - "stderr"
```

```
# 节点名称
name: "etcdnode1"
# 数据存储目录
data-dir: "/etcd-data/data"
# 预写式日志存储目录
wal-dir: "/etcd-data/wal"
# 集群成员之间通讯使用URL
listen-peer-urls: "http://0.0.0.0:12380"
# 集群提供给外部客户端访问的URL，即外部客户端必须通过指定的IP加端口访问etcd
listen-client-urls: "http://0.0.0.0:12379"

#集群配置
initial-advertise-peer-urls: "http://192.168.239.149:12380"
# 集群初始成员配置，是etcd静态部署的核心初始化配置，它说明了当前集群由哪些URLs组成,此处default为节点名称
initial-cluster:
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etcdnode2=http://192.168.239.149:22380"
# 初始化集群状态（new 或 existing）
initial-cluster-state: "new"
```

```
# 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
initial-cluster-token: "etcd-cluster"
# 向客户端发布的服务端点
advertise-client-urls: "http://192.168.239.149:12379"
logger: "zap"
# 配置日志级别,仅支持  debug, info, warn, error, panic, or fatal
log-level: "warn"
log-outputs:
  - "stderr"
```

```
# 节点名称
name: "etcdnode2"
# 数据存储目录
data-dir: "/etcd-data/data"
# 预写式日志存储目录
wal-dir: "/etcd-data/wal"
# 集群成员之间通讯使用URL
listen-peer-urls: "http://0.0.0.0:22380"
# 集群提供给外部客户端访问的URL，即外部客户端必须通过指定的IP加端口访问etcd
listen-client-urls: "http://0.0.0.0:22379"

#集群配置
initial-advertise-peer-urls: "http://192.168.239.149:22380"
# 集群初始成员配置，是etcd静态部署的核心初始化配置，它说明了当前集群由哪些URLs组成,此处default为节点名称
initial-cluster:
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etcdnode2=http://192.168.239.149:22380"
# 初始化集群状态（new 或 existing）
initial-cluster-state: "new"
# 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
initial-cluster-token: "etcd-cluster"
# 向客户端发布的服务端点
advertise-client-urls: "http://192.168.239.149:22379"
logger: "zap"
# 配置日志级别,仅支持  debug, info, warn, error, panic, or fatal
log-level: "warn"
log-outputs:
  - "stderr"
```

启动节点

1. 节点0

```
docker run -d -p 2379:2379 -p 2380:2380 -v /tmp/etcd0-data:/etcd-data   -v /home/nick/work/etcdconf:/etcd-conf --name etcd0 quay.io/coreos/etcd:v3.5.7 /usr/local/bin/etcd --config-file=/etcd-conf/etcd0.yaml
```

2. 节点1

```
docker run -d -p 12379:12379 -p 12380:12380 -v /tmp/etcd1-data:/etcd-data   -v /home/nick/work/etcdconf:/etcd-conf --name etcd1 quay.io/coreos/etcd:v3.5.7 /usr/local/bin/etcd --config-file=/etcd-conf/etcd1.yaml
```

3. 节点2

```
docker run -d -p 22379:22379 -p 22380:22380 -v /tmp/etcd2-data:/etcd-data   -v
/home/nick/work/etcdconf:/etcd-conf --name etcd2 quay.io/coreos/etcd:v3.5.7
/usr/local/bin/etcd --config-file=/etcd-conf/etcd2.yaml
```

验证

1. 验证集群，通过2379默认端口访问，无需指定--endpoints

```
docker exec etcd0 /usr/local/bin/etcdctl endpoint health --cluster -w table
```

```
nick@k8s-master:~$ docker exec etcd0 /usr/local/bin/etcdctl endpoint health --cluster -w table
+-----------------------------+--------+-------------+-------+
|          ENDPOINT           | HEALTH |    TOOK     | ERROR |
+-----------------------------+--------+-------------+-------+
| http://192.168.239.149:22379 |   true |  5.289297ms |       |
|  http://192.168.239.149:2379 |   true |  5.313698ms |       |
| http://192.168.239.149:12379 |   true | 10.681632ms |       |
+-----------------------------+--------+-------------+-------+
```

2. 验证集群，非默认端口访问，需指定--endpoints

```
docker exec etcd1 /usr/local/bin/etcdctl endpoint health --cluster -w table --
endpoints=[192.168.239.149:2379,192.168.239.149:12379,192.168.239.149:22379]
```

```
nick@k8s-master:~$ docker exec etcd1 /usr/local/bin/etcdctl endpoint health --cluster -w table --endpoints=[192.168.239.149:2379,192.168.239.149:12379,192.168.239.149:22379]
+-----------------------------+--------+------------+-------+
|          ENDPOINT           | HEALTH |    TOOK    | ERROR |
+-----------------------------+--------+------------+-------+
|  http://192.168.239.149:2379 |   true |  6.21939ms |       |
| http://192.168.239.149:22379 |   true | 9.021026ms |       |
| http://192.168.239.149:12379 |   true | 7.022993ms |       |
+-----------------------------+--------+------------+-------+
```

3. 验证单个节点

```
docker exec etcd1 /usr/local/bin/etcdctl endpoint health  -w table --
endpoints=192.168.239.149:22379
```

```
nick@k8s-master:~$ docker exec etcd0 /usr/local/bin/etcdctl endpoint health  -w table --endpoints=192.168.239.149:22379
+----------------------+--------+------------+-------+
|       ENDPOINT       | HEALTH |    TOOK    | ERROR |
+----------------------+--------+------------+-------+
| 192.168.239.149:22379 |   true | 3.932211ms |       |
+----------------------+--------+------------+-------+
nick@k8s-master:~$
```

```
# 若交互容器所在etcd 监听客户端端口为2379
docker exec etcd0 /usr/local/bin/etcdctl endpoint health --cluster -w table
# 监听客户端端口不为 2379，则需要指定endpoint
docker exec etcd1 /usr/local/bin/etcdctl endpoint health --cluster -w table --
endpoints=[192.168.239.149:2379,192.168.239.149:12379,192.168.239.149:22379]
```

```
nick@k8s-master:~/work/etcdconf$ docker exec -it etcd0 bash
root@4f0ddbe19f09:/# etcdctl endpoint health --cluster -w table
+-----------------------------+--------+-------------+-------+
|          ENDPOINT           | HEALTH |    TOOK     | ERROR |
+-----------------------------+--------+-------------+-------+
|  http://192.168.239.149:2379 |   true |  6.740167ms |       |
| http://192.168.239.149:22379 |   true |  9.212988ms |       |
| http://192.168.239.149:12379 |   true | 11.534708ms |       |
+-----------------------------+--------+-------------+-------+
root@4f0ddbe19f09:/#
```

etcdctl命令行

注意事项

1. 容器化部署后，需要通过docker exec 在容器中执行命令，使用容器内部的etcdctl命令行客户端

```
# 通过docker exec 在容器内部执行命令
docker exec etcd0 /usr/local/bin/etcdctl put key1 value1
```

2. etcdctl 默认访问当前节点2379端口，如果当前节点的etcd服务没有监听2379端口，那么需要指定--endpoints,可以指定某一个节点，或者集群所有节点

```
# etcdctl客户端访问监听了默认端口的节点
docker exec etcd0 /usr/local/bin/etcdctl put key1 value1
# etcdctl客户端访问某一个节点
docker exec  etcd1 /usr/local/bin/etcdctl put key1 value1 --
endpoints=192.168.239.149:22379
# etcdctl客户端访问集群，指定所有节点列表
docker exec  etcd1 /usr/local/bin/etcdctl put key1 value1 --endpoints=
[192.168.239.149:2379,192.168.239.149:12379,192.168.239.149:22379]
```

3. 容器化部署后，需要与终端保持连接的命令需要启用docker 虚拟终端交互（指定 -it 选项），例如：lock、elect、txn等

```
# 分布式锁，需要再多个终端执行
docker exec -it  etcd0 /usr/local/bin/etcdctl lock mutex1

# 分布式选主，需要在多个终端推选不同的候选人
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node1
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node2
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node3
```

```
# 开启事务
docker exec -it  etcd0 /usr/local/bin/etcdctl txn -i
```

全局参数

```
--cacert=""  #验证启用TLS的安全服务器的证书
--cert=""       # TLS证书文件
--command-timeout=5s # 运行命令超时时间
--debug[=false]             # 启用客户端调试日志记录
--dial-timeout=2s           # 客户端连接的拨号超时
-d, --discovery-srv=""      # 用于查询描述群集端点的SRV记录的域名
--discovery-srv-name=""     # 使用DNS发现时要查询的服务名称
--endpoints=[127.0.0.1:2379]    # gRPC 端点
--hex[=false]           # 将字节字符串打印为十六进制编码字符串
--insecure-discovery[=true]  # 接受描述群集端点的不安全SRV记录
--insecure-skip-tls-verify[=false]  # 跳过服务器证书验证（注意：此选项应仅用于测试目的）
--insecure-transport[=true]     # 禁用客户端连接的传输安全
--keepalive-time=2s         # 客户端连接的保持生存时间
--keepalive-timeout=6s      # 客户端连接的保持活动超时
```

```
--key=""                    # TLS密钥文件
--password=""           # 密码
--user=""           # 用户名
-w, --write-out="simple"            #设置输出格式 (fields, json, protobuf, simple,
table)
```

数据操作

put/get/del(数据读写)

```
put         #将给定的key写入到存储
    --ignore-lease[=false] #使用当前租约更新key
    --ignore-value[=false] #使用当前值更新key
    --lease="0" # 要附加到key的租约ID（十六进制）
    --prev-kv[=false] # 返回修改前的上一个键值对

del         #删除指定的键或键范围
    --from-key[=false] # 删除大于等于给定key的所有key（按byte值比较）
    --prefix[=false]        # 按前缀匹配删除
  --prev-kv[=false]     # 删除后是否返回被删除的键值对

get         #获取给定key或给定范围的key
    --consistency="l"   # 'l' 代表线性读（执行raft）， 's' 代表串行化读
    --count-only[=false] # 仅获取计数
    --from-key[=false] #使用byte值比较获取大于或等于给定键的键，类似*key*，但不等于
    --keys-only[=false] # 仅获取key
    --limit=0 #最大结果数
    --order="" #结果排序，ASCEND  或 DESCEND （默认 ASCEND）
    --prefix[=false] # 获取具有匹配前缀的key,key*
    --print-value-only[=false] #用“simple”输出格式时仅打印值
    --rev=0 #指定kv版本
    --sort-by="" # 指定排序字段，CREATE, KEY, MODIFY, VALUE, or VERSION
```

示例:

```
docker exec etcd0 /usr/local/bin/etcdctl put key1 value1
docker exec etcd0 /usr/local/bin/etcdctl put key2 value2
docker exec etcd0 /usr/local/bin/etcdctl put key3 value2

# 根据指定key获取
docker exec etcd0 /usr/local/bin/etcdctl get key1
# 根据前缀获取
docker exec etcd0 /usr/local/bin/etcdctl get key --prefix


docker exec etcd0 /usr/local/bin/etcdctl put a valuea
docker exec etcd0 /usr/local/bin/etcdctl put b valueb
docker exec etcd0 /usr/local/bin/etcdctl put c valuec

# 根据key值byte值比较，获取大于等于指定key值的所有key
docker exec etcd0 /usr/local/bin/etcdctl get b --from-key
```

lease(租约)

```
lease grant     # 创建租约
lease keep-alive    # 开启线程，自动续约
    --once[=false]  # 仅续约一次

lease list      #列出所有活动的租约
lease revoke        #撤销租约
lease timetolive    # 获取租约信息
    --keys[=false] #获取附加到此租约的key
```

示例:

```
# 创建租约，有效时长20s
docker exec etcd0 /usr/local/bin/etcdctl lease grant 20
# 获取租约列表
docker exec etcd0 /usr/local/bin/etcdctl lease list
# 根据leaseID 撤销租约
docker exec etcd0 /usr/local/bin/etcdctl lease revoke 694d869b526b302a
# 查询指定租约的有效时长信息
docker exec etcd0 /usr/local/bin/etcdctl lease timetolive 694d869b526b30c0
# 开启线程为指定租约自动续租
docker exec etcd0 /usr/local/bin/etcdctl lease keep-alive 694d869b526b30c2
```

lock(锁)

```
lock            # 获取命名锁
    --ttl=10        #session 超时时间
```

示例1:

```
# 分布式锁，需要再多个终端执行
# 终端1
docker exec -it  etcd0 /usr/local/bin/etcdctl lock mutex1
# 终端2
docker exec -it  etcd0 /usr/local/bin/etcdctl lock mutex1
```

示例2:

```
tx01 $ etcdctl --endpoints=$ENDPOINTS
```

gyuho@tx01: ~ 67x6

```
tx01 $ etcdctl --endpoints=$ENDPOINTS
```

elect (选举)

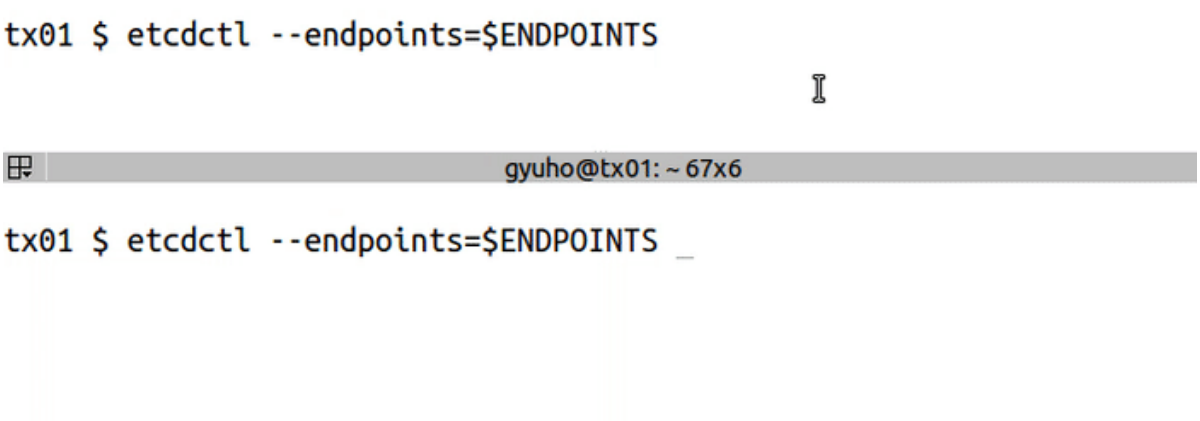
选举逻辑与锁逻辑类似，前一个leader卸任之后，后续竞选者才能选出新的leader

```
elect #观察并参与领导人选举
     -l, --listen[=false]   # 观察模式
```

```
etcdctl elect <election-name> [proposal] [flags]
```

```
# 分布式选主，需要在多个终端推选不同的候选人
# 终端1
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node1
# 终端2
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node2
# 终端3
docker exec -it  etcd0 /usr/local/bin/etcdctl elect myelect node3
```

示例：



txn(事务)

```
txn          # 在一个事务中处理所有请求
    -i, --interactive[=false] # 交互模式下的输入事务
```

事务API由 if 语句、then语句、else语句组成。基本逻辑：在 If 语句中，可以添加一系列的条件表达式，若条件表达式全部通过检查，则执行 Then 语句的 get/put/delete 等操作，否则执行 Else 的 get/put/delete 等操作，支持比较运算符 （>、<、=、!=）
if语句支持项：

1. key 的最近一次修改版本号 mod_revision，简称 mod，可以用于检查 key 最近一次被修改时的版本号是否符合你的预期。例如：mod("key1") = "111902"，检查key1最近一次修改版本号是否等于111902
2. key 的创建版本号 create_revision，简称 create,可以用于检测 key 是否已存在。例如：create("key") = "0"，来判断key1是否存在，不存在的话 create_revision 版本号就是0。
3. key 的修改次数 version；可以用于检查 key 的修改次数是否符合预期。例如：version("key1") > "3",来判断key1的修改次数是否大于3次
4. key 的值，可以用于检查 key 的 value 值是否符合预期。例如：value("key1") = "value1"，来判断key1 的值是否等于 value1

示例1：

```
tx01 $ etcdctl --endpoints=$ENDPOINTS put user1 bad
OK
tx01 $ etcdctl --endpoints=$ENDPOINTS _
```

```
tx01 $ etcdctl --endpoints=$ENDPOINTS txn --interactive
compares:          if
value("user1") = "bad"

success requests (get, put, delete): then
del user1

failure requests (get, put, delete):    else
put user1 good

SUCCESS

1
tx01 $
```

示例2：

```
nick@k8s-master:~$ docker exec -it  etcd0 /usr/local/bin/etcdctl txn -i
compares:
create("key2") = "0"

success requests (get, put, del):
put key2 value2

failure requests (get, put, del):
get key2

SUCCESS

OK
crate^Cnick@k8s-master:~$ docker exec -it  etcd0 /usr/local/bin/etcdctl txn -i
compares:
create("key2") = "0"

success requests (get, put, del):
put key2 value2

failure requests (get, put, del):
get key2

FAILURE

key2
value2
```

watch(事件监听)

```
watch           #监听键值或前缀的事件流
    -i, --interactive[=false] # 交互式模式
    --prefix[=false]    # 监听前缀
    --prev-kv[=false] # 获取事件发生前的上一个键值对
    --progress-notify[=false] # 从服务器获取定期监视进度通知
    --rev=0 # 从指定版本开始监听
```

```
# 终端1-监听key2
docker exec etcd0 /usr/local/bin/etcdctl watch key2
# 终端2-对key2做操作
docker exec  etcd0 /usr/local/bin/etcdctl put key2 value3
docker exec  etcd0 /usr/local/bin/etcdctl put key2 value4
docker exec  etcd0 /usr/local/bin/etcdctl del key2
```

```
nick@k8s-master:~$ docker exec etcd0 /usr/local/bin/etcdctl watch key2
PUT
key2
value3
PUT
key2
value4
DELETE
key2
```

身份认证

auth

```
#禁用身份认证
auth disable
#启用身份认证
auth enable
# 返回身份认证状态
auth status
```

```
# 查看身份认证状态
docker exec  etcd0 /usr/local/bin/etcdctl auth status
# 启用身份认证，没有添加root用户之前，无法启用
docker exec  etcd0 /usr/local/bin/etcdctl auth enable
# 禁用身份认证，启用之后再禁用需要提供用户名和密码，如： --user="root" --password="123456"
docker exec  etcd0 /usr/local/bin/etcdctl auth disable
```

role(角色)

```
#添加新角色
role add
#删除角色
role delete
#获取角色的详细信息
role get
#向角色授予key
```

```
role grant-permission
    --from-key[=false] # 使用key值的byte比较获取大于或等于给定键的键授权给角色，类似
*key*，但不等于
    --prefix[=false] # 授予前缀授权
#列出所有角色
role list
#从角色吊销key
role revoke-permission
    # 根据key的byte值比较吊销权限
    --from-key[=false]
  # 吊销前缀权限
    --prefix[=false]
```

```
# 添加角色
docker exec  etcd0 /usr/local/bin/etcdctl role add teacher
docker exec  etcd0 /usr/local/bin/etcdctl role add student
# 获取角色列表
docker exec  etcd0 /usr/local/bin/etcdctl role list
# 授予角色权限
# 给teacher角色授予所有key的读写权限
docker exec  etcd0 /usr/local/bin/etcdctl role grant-permission --prefix teacher
readwrite  ""
# 给student角色授予所有key的读权限
docker exec  etcd0 /usr/local/bin/etcdctl role grant-permission --prefix student
read  ""
# 查看角色详情
docker exec  etcd0 /usr/local/bin/etcdctl role get student
# 吊销授权
docker exec  etcd0 /usr/local/bin/etcdctl role revoke-permission --prefix student
""
# 删除角色
docker exec  etcd0 /usr/local/bin/etcdctl role delete student
```

user(用户)

```
user add #添加新用户
    --interactive[=true]    # 从标准输入读取密码，而不是交互终端
  --new-user-password=""    # 通过命令行flag设置密码
  --no-password[=false] # 创建不带密码的用户（仅限基于CN的身份验证）

user delete #删除用户

user get #获取用户的详细信息
    --detail[=false] # 显示授予用户的角色权限

user grant-role #向用户授予角色

user list #列出所有用户

user passwd #更改用户的密码
    --interactive[=true]    # 从标准输入读取密码，而不是交互终端
user revoke-role #从用户撤消角色
```

```
# 创建用户
docker exec etcd0 /usr/local/bin/etcdctl user add nick --new-user-
password="123456"
# 授予用户角色
docker exec  etcd0 /usr/local/bin/etcdctl user grant-role nick teacher
# 获取用户信息
docker exec  etcd0 /usr/local/bin/etcdctl user get nick
# 列出所有用户
docker exec etcd0 /usr/local/bin/etcdctl user list
# 撤销用户角色
docker exec etcd0 /usr/local/bin/etcdctl user revoke-role nick teacher
# 删除用户
docker exec  etcd0 /usr/local/bin/etcdctl user delete nick
```

```
# 由于容器交互暂无法解决密码修改的问题，采用宿主机本地etcdctl 命令行客户端访问集群
./etcdctl --endpoints=192.168.239.149:22379 user passwd nick
```

启用身份认证

```
# 添加root角色
docker exec  etcd0 /usr/local/bin/etcdctl role add root
# 创建root用户
docker exec etcd0 /usr/local/bin/etcdctl user add root --new-user-
password="123456"
# 向root用户授予root角色
docker exec  etcd0 /usr/local/bin/etcdctl user grant-role root root --user="root"
--password="123456"
# 启用身份认证
docker exec  etcd0 /usr/local/bin/etcdctl auth enable
```

运维

alarm(报警)

```
#解除所有报警
alarm disarm
# 列出所有报警
alarm list
```

```
# 获取集群警告列表
docker exec  etcd0 etcdctl alarm list
# 解除集群所有警告
docker exec  etcd0 etcdctl alarm disarm
```

check(检查)

```
check datascale #   检查给定服务器端点不同工作负载下的内存使用情况
    --auto-compact[=false]          # 测试完成后，用最新版本压缩存储
  --auto-defrag[=false]         # 测试完成后对存储进行碎片整理
  --load="s"                # 接受的工作负载：s（小）、m（中）、l（大）、xl（xLarge）
  --prefix="/etcdctl-check-datascale/"  # 写入datascale检查键的前缀

check perf      # 检查etcd群集的性能
    --auto-compact[=false]          # 测试完成后，用最新版本压缩存储
  --auto-defrag[=false]         # 测试完成后对存储进行碎片整理
  --load="s"                # 接受的工作负载：s（小）、m（中）、l（大）、xl（xLarge）
  --prefix="/etcdctl-check-datascale/"  # 写入datascale检查键的前缀
```

1. check datascale 示例

```
# 检查端口为2379 的服务器端点的内存使用情况
docker exec  etcd0 etcdctl check datascale  --load="s" --prefix="/etcdctl-check-datascale/"
# 检查端口为12379与22379 的服务器端点的内存使用情况
docker exec  etcd0  etcdctl check datascale --load="s" --prefix="/etcdctl-check-datascale/" --endpoints=192.168.239.149:12379,192.168.239.149:22379
```

2. check perf 示例

```
# 检查集群性能
docker exec  etcd0 etcdctl check perf --load="s" --prefix="/etcdctl-check-datascale/"
```

compaction(压缩)

```
compaction # 压缩etcd中的事件历史记录
    --physical[=false] # “true”表示等待压缩以物理删除所有旧版本
```

```
etcdctl compaction [options] <revision> [flags]
```

```
# 压缩版本号为59010 之前的所有版本，即删除59010之前的所有版本
docker exec etcd0 etcdctl compaction --physical 59010
```

defrag(碎片整理)

```
defrag #对具有给定端点的etcd成员的存储进行碎片整理
    --cluster[=false] # 使用群集成员列表中的所有端点
    --data-dir="" #可选择的如果存在，对etcd未使用的数据目录进行碎片整理
```

示例：

```
etcdctl defrag
etcdctl defrag --endpoints=192.168.239.149:22379
```

endpoint(端点)

```
endpoint hashkv #打印--endpoint中每个端点的KV历史哈希
    --rev=0 # 最大版本的哈希（默认：所有版本）

endpoint health #检查“--endpoints”标志中指定的终结点的健康状况

endpoint status #打印“--endpoints”标志中指定的端点的状态
```

```
docker exec etcd0 /usr/local/bin/etcdctl endpoint hashkv --cluster
docker exec etcd0 /usr/local/bin/etcdctl endpoint hashkv --
endpoints=192.168.239.149:22379

docker exec etcd0 /usr/local/bin/etcdctl endpoint health --cluster
docker exec etcd0 /usr/local/bin/etcdctl endpoint health --
endpoints=192.168.239.149:22379

docker exec etcd0 /usr/local/bin/etcdctl endpoint status --cluster  -w table
docker exec etcd0 /usr/local/bin/etcdctl endpoint status --
endpoints=192.168.239.149:22379 -w table
```

member(成员)

```
member add #将成员添加到群集中
    --learner[=false]   # 表示是否将节点标识为 raft learner(只读不参与leader选举)
    --peer-urls="" # 新成员的逗号分隔对等URL

member list #列出群集中的所有成员

# 将无投票权的 learner 成员，提升为集群中有投票权的成员
member promote

member remove #从群集中删除成员

member update #更新群集中的成员
    --peer-urls="" #更新成员的逗号分隔成员URL
```

示例配置文件:

```
# 节点名称
name: "etcdnode3"
# 数据存储目录
data-dir: "/etcd-data/data"
# 预写式日志存储目录
wal-dir: "/etcd-data/wal"
# 集群成员之间通讯使用URL
listen-peer-urls: "http://0.0.0.0:32380"
# 集群提供给外部客户端访问的URL，即外部客户端必须通过指定的IP加端口访问etcd
listen-client-urls: "http://0.0.0.0:32379"
```

```
#集群配置
initial-advertise-peer-urls: "http://192.168.239.149:32380"
# 集群初始成员配置，是etcd静态部署的核心初始化配置，它说明了当前集群由哪些URLs组成,此处default为节点名称
initial-cluster: "etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etcdnode2=http://192.168.239.149:22380,etcdnode3=http://192.168.239.149:32380"
# 初始化集群状态（new 或 existing）
initial-cluster-state: "existing"
# 引导期间etcd集群的初始集群令牌,防止不同集群之间产生交互
initial-cluster-token: "etcd-cluster"
# 向客户端发布的服务端点
advertise-client-urls: "http://192.168.239.149:32379"
logger: "zap"
# 配置日志级别,仅支持  debug, info, warn, error, panic, or fatal
log-level: "warn"
log-outputs:
  - "stderr"
```

```
# 添加节点到现有集群，先添加节点，再启动节点
docker exec etcd0 etcdctl member add etcd3 --peer-urls="http://192.168.239.149:32380"
# 启动新的节点
docker run -d -p 32379:32379 -p 32380:32380 -v /tmp/etcd3-data:/etcd-data   -v /home/nick/work/etcdconf:/etcd-conf --name etcd3 quay.io/coreos/etcd:v3.5.7 /usr/local/bin/etcd --config-file=/etcd-conf/etcd3.yaml
```

```
# 获取节点列表
docker exec etcd0 etcdctl member list -w table
# 从集群中移除节点
docker exec etcd0 etcdctl member remove a05dc0d224e70ad8
```

move-leader(移交leader)

```
move-leader #将领导权转移到另一个etcd集群成员
```

```
etcdctl move-leader <transferee-member-id> [flags]
```

```
# 查看集群状态
docker exec etcd0 etcdctl endpoint status --cluster  -w table
# 移交leader权限
docker exec etcd0 etcdctl move-leader 3054d0c5a09f14e8
# 查看集群状态
docker exec etcd0 etcdctl endpoint status --cluster  -w table
```

make-mirror(镜像)

```
# 在另外一个集群构建一个镜像。
# 同时实时将一个集群的变更同步到另外一个集群中。
# 同时会在终端上打印同步的key个数
make-mirror # 在目标etcd群集上创建镜像
```

```
etcdctl make-mirror [options] <destination> [flags]
```

```
--dest-cacert=""            Verify certificates of TLS enabled secure servers
using this CA bundle
--dest-cert=""          Identify secure client using this TLS certificate file
for the destination cluster
--dest-insecure-transport[=true]    Disable transport security for client
connections
--dest-key=""           Identify secure client using this TLS key file
--dest-password=""      Destination password for authentication (if this option
is used, --user option shouldn't include password)
--dest-prefix=""            destination prefix to mirror a prefix to a different
prefix in the destination cluster
--dest-user=""          Destination username[:password] for authentication
(prompt if password is not supplied)
--no-dest-prefix[=false]        mirror key-values to the root of the destination
cluster
--prefix=""         Key-value prefix to mirror
```

```
# 此处以当前集群镜像到当前集群
# 将在当前集群构建镜像，并且将key为前缀的数据镜像到yek为前缀
docker exec etcd0 etcdctl make-mirror http://192.168.239.149:22379 --prefix="key"
--dest-prefix="yek"
# 集群发生变更会产生同步
docker exec etcd0 etcdctl put key3 value3
```

snapshot (快照)

数据快照

```
snapshot save #将etcd节点后端快照存储到给定文件
snapshot restore #将etcd成员快照还原到etcd目录
    --data-dir string #数据目录
    --initial-advertise-peer-urls string    #要向群集中其他成员通告的此成员的成员URL列表
    --initial-cluster string #还原引导的初始群集配置
    --initial-cluster-token string #还原引导期间etcd群集的初始群集令牌
    --name string # 成员名称
    --skip-hash-check #忽略快照完整性哈希值（如果从数据目录复制，则为必需）
    --wal-dir string #预写式日志目录
snapshot status #获取给定文件的后端快照状态
```

注意使用方式：etcdctl 与 etcdutl

```
# etcdctl 命令
etcdctl snapshot save <filename> [flags]

# etcdutl 命令
etcdutl snapshot restore <filename> --data-dir {output dir} [options] [flags]

# etcdutl 命令
etcdutl snapshot status <filename> [flags]
```

```
# 保存镜像到文件
docker exec etcd0 etcdctl snapshot save /etcd-snapshot
# 将文件从容器中拷贝到宿主机
docker cp etcd0:/etcd-snapshot ./
# 查看镜像文件状态
docker exec etcd0 etcdutl snapshot status /etcd-snapshot
```

从镜像恢复etcd

1. 恢复之前插入一条数据，用于恢复前后对比

```
# 恢复之前插入一条数据，用于恢复前后对比
docker exec etcd0 etcdctl put beforrestore restorevalue
```

2. 启动一个容器用于辅助，将集群的所有节点的数据目录全部挂载到容器

```
docker run -d \
  -p 42379:42379 \
  -p 42380:42380 \
  --mount type=bind,source=/tmp/etcd-data.tmp,destination=/etcd-data \
    -v /tmp/etcd0-data:/etcd0-data \
    -v /tmp/etcd1-data:/etcd1-data \
    -v /tmp/etcd2-data:/etcd2-data \
    -v /tmp/etcd3-data:/etcd3-data \
  --name etcd-gcr-v3.5.7 \
  quay.io/coreos/etcd:v3.5.7 \
  /usr/local/bin/etcd \
  --name s1 \
  --data-dir /etcd-data \
  --listen-client-urls http://0.0.0.0:42379 \
  --advertise-client-urls http://0.0.0.0:42379 \
  --listen-peer-urls http://0.0.0.0:42380 \
  --initial-advertise-peer-urls http://0.0.0.0:42380 \
  --initial-cluster s1=http://0.0.0.0:42380 \
  --initial-cluster-token tkn \
  --initial-cluster-state new \
  --log-level info \
  --logger zap \
  --log-outputs stderr
```

3. 将备份文件拷贝到辅助容器

```
docker cp ./etcd-snapshot etcd-gcr-v3.5.7:/etcd-snapshot
```

4. 数据必须恢复到一个空目录，因此我们必须在恢复后为集群节点重新指定数据目录。

（此处为了复用配置文件和启动命令，我们将原有集群的数据目录进行清空）

```
# 删除现有集群的所有容器
docker rm -f etcd0 etcd1 etcd2 etcd3
# 清空目录
sudo rm -r /tmp/etcd0-data/* /tmp/etcd1-data/* /tmp/etcd2-data/* /tmp/etcd3-
data/*
```

5. 此处开始通过辅助容器恢复数据到指定目录

```
# 恢复节点0的数据
docker exec etcd-gcr-v3.5.7 etcdutl snapshot restore /etcd-snapshot \
--data-dir /etcd0-data/data --wal-dir /etcd0-data/wal \
--initial-advertise-peer-urls="http://192.168.239.149:2380" \
--initial-cluster
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etc
dnode2=http://192.168.239.149:22380,etcdnode3=http://192.168.239.149:32380" \
--initial-cluster-token etcd-cluster --name etcdnode0

# 恢复节点1的数据
docker exec etcd-gcr-v3.5.7 etcdutl snapshot restore /etcd-snapshot \
--data-dir /etcd1-data/data --wal-dir /etcd1-data/wal \
--initial-advertise-peer-urls="http://192.168.239.149:12380" \
--initial-cluster
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etc
dnode2=http://192.168.239.149:22380,etcdnode3=http://192.168.239.149:32380" \
--initial-cluster-token etcd-cluster --name etcdnode1

# 恢复节点2的数据
docker exec etcd-gcr-v3.5.7 etcdutl snapshot restore /etcd-snapshot \
--data-dir /etcd2-data/data --wal-dir /etcd2-data/wal \
--initial-advertise-peer-urls="http://192.168.239.149:22380" \
--initial-cluster
"etcdnode0=http://192.168.239.149:2380,etcdnode1=http://192.168.239.149:12380,etc
dnode2=http://192.168.239.149:22380,etcdnode3=http://192.168.239.149:32380" \
--initial-cluster-token etcd-cluster --name etcdnode2
```

6. 重新创建集群

```
docker run -d -p 2379:2379 -p 2380:2380 -v /tmp/etcd0-data:/etcd-data   -v
/home/nick/work/etcdconf:/etcd-conf --name etcd0 quay.io/coreos/etcd:v3.5.7
/usr/local/bin/etcd --config-file=/etcd-conf/etcd0.yaml

docker run -d -p 12379:12379 -p 12380:12380 -v /tmp/etcd1-data:/etcd-data   -v
/home/nick/work/etcdconf:/etcd-conf --name etcd1 quay.io/coreos/etcd:v3.5.7
/usr/local/bin/etcd --config-file=/etcd-conf/etcd1.yaml

docker run -d -p 22379:22379 -p 22380:22380 -v /tmp/etcd2-data:/etcd-data   -v
/home/nick/work/etcdconf:/etcd-conf --name etcd2 quay.io/coreos/etcd:v3.5.7
/usr/local/bin/etcd --config-file=/etcd-conf/etcd2.yaml
```

7. 检查集群

```
# 查看集群节点健康状态
docker exec etcd0 etcdctl  endpoint health --cluster -w table
# 对比数据
docker exec etcd0 etcdctl get "" --prefix
```

grpc-proxy

1. 文档:

https://etcd.io/docs/v3.5/op-guide/grpc_proxy/
https://etcd.io/docs/v3.5/dev-guide/grpc_naming/
grpc-proxy是在 gRPC 层 （L7） 运行的无状态 etcd 反向代理。该代理旨在降低etcd集群的负载。可水平伸缩，合并了watch请求和lease请求。缓存了客户连接防止客户端滥用，以保护etcd集群。
watch 请求的合并

```
             +-------------+
             | etcd server |
             +------+------+
                    ^ watch key A (s-watcher)
                    |
            +-------+-----+
            | gRPC proxy  | <-------+
            |             |         |
            ++-----+------+         |watch key A (c-watcher)
 watch key A ^     ^ watch key A    |
 (c-watcher) |     | (c-watcher)    |
     +-------+-+  ++--------+  +----+----+
     |  client |  |  client |  |  client |
     |         |  |         |  |         |
     +---------+  +---------+  +---------+
```

lease 请求的合并

```
           +-------------+
           | etcd server |
           +------+------+
                  ^
                  | heartbeat L1, L2, L3
                  | (s-stream)
                  v
          +-------+-----+
          | gRPC proxy  +<-----------+
          +---+------+--+            | heartbeat L3
              ^      ^               | (c-stream)
 heartbeat L1 |      | heartbeat L2  |
 (c-stream)   v      v (c-stream)    v
       +------+-+  +-+------+  +-----+--+
       | client |  | client |  | client |
       +--------+  +--------+  +--------+
```

gprc-proxy 支持多个etcd服务器端点。当代理启动时，随机选择一个服务器端点，将所有请求转发到该端点。如果端点出现故障则会切换到其他端点，以对其客户端隐藏故障

2. 通过--namespace指定命名空间，为所有通过该代理的key附件前缀
3. 代理集群：1.允许客户端根据设置的端点同步其他端点；2.可用作grpc名称解析器

```
docker run -d -p 23791:23791 --name etcd-grpc-proxy-resolver1 \
quay.io/coreos/etcd:v3.5.7 etcd grpc-proxy start  \
--namespace=/my-prefix/ \
--listen-addr=0.0.0.0:23791 \
--advertise-client-url=192.168.239.149:23791 \
--resolver-prefix="___grpc_proxy_endpoint" \
--resolver-ttl=10 \
--
endpoints=http://192.168.239.149:2379,http://192.168.239.149:12379,http://192.168
.239.149:22379
```

```
docker run -d -p 23792:23792 --name etcd-grpc-proxy-resolver2 \
quay.io/coreos/etcd:v3.5.7 etcd grpc-proxy start  \
--namespace=/my-prefix/ \
--listen-addr=0.0.0.0:23792 \
--advertise-client-url=192.168.239.149:23792 \
--resolver-prefix="___grpc_proxy_endpoint" \
--resolver-ttl=10 \
--
endpoints=http://192.168.239.149:2379,http://192.168.239.149:12379,http://192.168
.239.149:22379
```

```
docker run -d -p 23793:23793 --name etcd-grpc-proxy-resolver3 \
quay.io/coreos/etcd:v3.5.7 etcd grpc-proxy start  \
--namespace=/my-prefix/ \
--listen-addr=0.0.0.0:23793 \
--advertise-client-url=192.168.239.149:23793 \
--resolver-prefix="___grpc_proxy_endpoint" \
--resolver-ttl=10 \
--
endpoints=http://192.168.239.149:2379,http://192.168.239.149:12379,http://192.168
.239.149:22379
```

```
docker exec etcd-grpc-proxy-resolver1 etcdctl  \
--endpoints=192.168.239.149:23791 put key1 value1

docker exec etcd-grpc-proxy-resolver1 etcdctl  \
--endpoints=192.168.239.149:23791 member list \
--write-out table
```

gateway

1. 文档：https://etcd.io/docs/v3.5/op-guide/gateway/

etcd 网关是一个简单的 TCP 代理，它将网络数据转发到 etcd 集群。网关是无状态且透明的;它既不检查客户端请求，也不干扰群集响应。它不会终止 TLS 连接，不会代表其客户端执行 TLS 握手，也不会验证连接是否安全。
网关支持多个 etcd 服务器端点，并采用简单的轮询策略。它仅路由到可用终结点，并对其客户端隐藏故障

2. 网关的使用场景，解耦应用程序与etcd集群端点。应用程序访问本地etcd网关程序，网关程序则访问etcd集群。当集群发生变化时，只需要调整网关即可，无需调整应用程序。网关程序只支持请求转发，不能提升访问性能
3. 启动网关

```
docker run -d -p 23790:23790 --name etcd-gateway \
quay.io/coreos/etcd:v3.5.7 etcd gateway start  \
--listen-addr=0.0.0.0:23790 \
--endpoints=192.168.239.149:23791,192.168.239.149:23792,192.168.239.149:23793
```

```
# 通过网关访问集群，网关访问集群
docker exec etcd-gateway etcdctl --endpoints=127.0.0.1:23790 put key1 value1
```

go客户端接入

1. 依赖安装

```
go get go.etcd.io/etcd/client/v3@v3.5.9
google.golang.org/grpc@v1.58.0
```

2. proto文件声明

```
syntax = "proto3";

package echo;
option go_package="etcd/echo";
import "google/protobuf/timestamp.proto";

message EchoRequest{
  string message = 1;
}
message EchoResponse{
  string message = 1;
}
message EchoLockResponse{
  int64 id = 1;
  string message = 2;
  google.protobuf.Timestamp start = 3;
  google.protobuf.Timestamp end = 4;
}

service Echo{
  rpc UnaryEcho(EchoRequest) returns (EchoResponse){}
  rpc UnaryLock(EchoRequest) returns (EchoLockResponse){}
}
```

3. 代码生成

```
protoc.exe --go_out=. --go_opt=paths=source_relative \
--go-grpc_out=. --go-grpc_opt=paths=source_relative .\echo\echo.proto
```